МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Департамент цифровых, робототехнических систем и электроники

«Модули и пакеты»
Отчет по лабораторной работе № 1
по дисциплине «Объектно-ориентированное программирование»
Вариант 5

Выполнил студент группы ИВТ-б-о-24-1
Грабарь Артемий Павлович
«__» сентября 2026г.
Подпись студента_____
Работа защищена « »_____20__г.
Проверил Воронкин Р.А. _____
(подпись)

Ставрополь 2026

Цель работы: приобретение навыков по работе с модулями и пакетами языка программирования Python версии 3.x.

Ссылка на GitHub: https://github.com/Arhi258/OOP_Laba_1.git

Задание:

1. Создать общедоступный репозиторий на GitHub, в котором будет использована лицензия MIT и выбранный язык программирования.
2. Выполнить клонирование созданного репозитория на рабочий компьютер.
3. Дополнить файл .gitignore необходимыми правилами для работы с IDE PyCharm.
4. Создать проект PyCharm в папке репозитория
5. Решить практические задания работы.
6. Зафиксировать сделанные изменения в репозитории.
7. Отправить сделанные изменения на сервер GitHub.

Ход работы:

Был создан общедоступный репозиторий на GitHub с лицензией MIT.

Было выполнено клонирование репозитория на рабочий компьютер, после чего в его каталоге был открыт проект PyCharm.

Были настроены файлы README.md и .gitignore, создано виртуальное окружение .venv, установлены инструменты pytest и Ruff. Для управления зависимостями и настройками проверок использованы файлы requirements.txt и pyproject.toml.

Было выполнено задание 1: «Используя замыкания функций, объявите внутреннюю функцию, которая принимает в качестве параметров фамилию и имя, а затем, заносит в шаблон эти данные. Сам шаблон - это строка, которая передается внешней функции и, например, может иметь такой вид: «Уважаемый %F%, %N%! Вы делаете работу по замыканиям функций.» Здесь %F% - это фрагмент куда нужно подставить фамилию, а %N% - фрагмент, куда нужно подставить имя. Здесь важно, чтобы внутренняя функция умела подставлять данные в шаблон, формировать новую строку и возвращать

результат. Вызовите внутреннюю функцию замыкания и отобразите на экране результат ее работы.». Результат выполнения работы представлен на рисунке 1. Программный код задания представлен в приложении 1.

Для выполнения первого задания функция замыкания была размещена в отдельном пакете closure. В файле module.py была реализована подстановка фамилии и имени в текстовый шаблон; файл __init__.py обеспечивает импорт функции, а main.py — ввод данных и вывод результата.

```
Введите фамилию: Грабарь
Введите имя: Артемий
Уважаемый Грабарь Артемий! Вы делаете работу по замыканию функций.
```

Рисунок 1. Результат выполнения задания 1

Было выполнено задание 2: «Решить индивидуальное задание лабораторной работы 2.6, оформив каждую команду в виде отдельной функции. Использовать словарь, содержащий следующие ключи: название пункта назначения рейса; номер рейса; тип самолета. Написать программу, выполняющую следующие действия: ввод с клавиатуры данных в список, состоящий из словарей заданной структуры; записи должны быть размещены в алфавитном порядке по названиям пунктов назначения; вывод на экран пунктов назначения и номеров рейсов, обслуживаемых самолетом, тип которого введен с клавиатуры; если таких рейсов нет, выдать на дисплей соответствующее сообщение.». Результат выполнения задания представлен на рисунке 2. Программный код задания представлен в приложении 2.

Для выполнения второго задания были созданы отдельные пакеты input_flights, sort_flights, print_flights и find_flights, каждый из которых содержит свою функцию. Через файлы __init__.py организован импорт функций, а в main.py обеспечен последовательный запуск операций ввода, сортировки, вывода и поиска рейсов.

```
Введите количество рейсов: 3
Введите пункт назначения: Москва
Введите номер рейса: 12
Введите тип самолета: ТУ-13
Введите пункт назначения: Город
Введите номер рейса: 1
Введите тип самолета: ТЦ-1
Введите пункт назначения: Сызрань
Введите номер рейса: 3
Введите тип самолета: ПП-02

Список рейсов:
Город 1 ТЦ-1
Москва 12 ТУ-13
Сызрань 3 ПП-02

Введите тип самолета для поиска: ПП-02
Пункт назначения: Сызрань, номер рейса: 3

Process finished with exit code 0
```

Рисунок 2. Результат выполнения задания 2

Была выполнена проверка программного кода с использованием Ruff. Результат анализа показан на рисунке 3, ошибок при проверке не обнаружено.

```
PS Z:\Репозиторий\OOP_Laba_1> .\.venv\Scripts\python.exe -m ruff check .
All checks passed!
```

Рисунок 3. Результат проверки программного кода с использованием Ruff

Затем была выполнена проверка форматирования программных файлов. По результатам проверки 18 файлов соответствовали требованиям форматирования, что представлено на рисунке 4.

```
PS Z:\Репозиторий\OOP_Laba_1> .\.venv\Scripts\python.exe -m ruff format --check .
18 files already formatted
```

Рисунок 4. Результат проверки форматирования программного кода

Для проверки корректности разработанных функций были подготовлены автоматические тесты с использованием pytest. Проверялись

подстановка текста в шаблон, сортировка рейсов, ввод и вывод данных, а также поиск рейсов. Все 10 тестов завершились успешно. Результат представлен на рисунке 5.

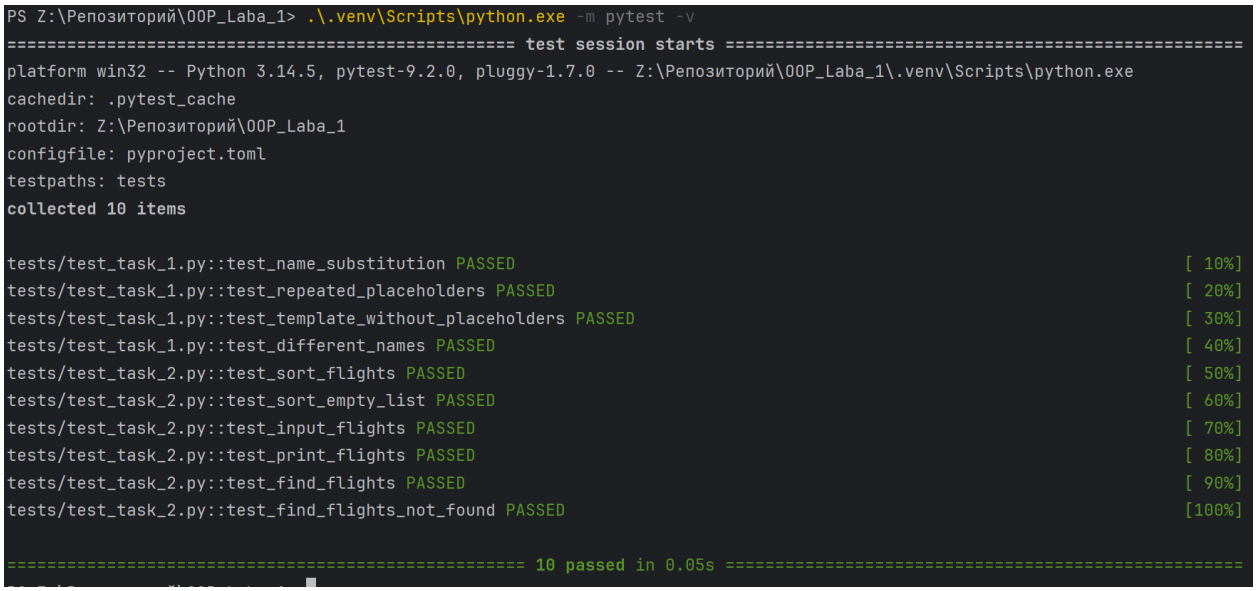


Рисунок 5. Результат автоматического тестирования с использованием pytest

Изменения в проекте были зафиксированы с помощью Git и отправлены в удалённый репозиторий на GitHub.

Контрольные вопросы:

1. Модуль в Python – это файл с расширением .py, содержащий программный код, функции, классы и переменные.

2. Для импорта модуля используется конструкция import имя_модуля. Для импорта отдельного объекта применяется from имя_модуля import имя_объекта.

3. Пакет – это каталог, объединяющий модули и другие пакеты. В рассматриваемом проекте пакеты содержат файлы __init__.py.

4. Файл __init__.py служит для инициализации пакета при его импорте; в нём могут находиться инструкции импорта и определения доступных объектов.

5. Переменная __all__ задаёт перечень имён, экспортируемых при использовании конструкции from package import *.

Вывод: в ходе лабораторной работы были получены навыки создания и использования модулей и пакетов Python, организации функций в отдельных пакетах, настройки виртуального окружения, автоматического тестирования и проверки оформления программного кода.

Приложение

Приложение 1. Программный код задания 1
Код файла Tasks/task_1/closure/__init__.py

```
from .module import closure


__all__ = ["closure"]
```

Код файла Tasks/task_1/closure/module.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-



def closure(template):
    def make_message(surname, name):
        result = template.replace("%F%", surname)
        result = result.replace("%N%", name)
        return result


    return make_message
```

Код файла Tasks/task_1/__init__.py

```
from .closure import closure


__all__ = ["closure"]
```

Код файла Tasks/task_1/main.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-


from .closure import closure


if __name__ == "__main__":
```

line = "Уважаемый %F% %N%! Вы делаете работу по замыканию функций."

message = closure(line)

surname = input("Введите фамилию: ")
name = input("Введите имя: ")

print(message(surname, name))

Приложение 2. Программный код задания 2
Код файла Tasks/task_2/input_flights/__init__.py

```
from .module import input_flights

__all__ = ["input_flights"]
```

Код файла Tasks/task_2/input_flights/module.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-


def input_flights():
    flights = []
    count = int(input("Введите количество рейсов: "))

    for _ in range(count):
        destination = input("Введите пункт назначения: ")
        flight_number = input("Введите номер рейса: ")
        plane_type = input("Введите тип самолета: ")

        flight = {
            "destination": destination,
            "flight_number": flight_number,
            "plane_type": plane_type,
        }
```

```
        flights.append(flight)


    return flights
```

Код файла Tasks/task_2/sort_flights/__init__.py

```
from .module import sort_flights


__all__ = ["sort_flights"]
```

Код файла Tasks/task_2/sort_flights/module.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-



def sort_flights(flights):
    flights.sort(key=lambda flight: flight["destination"].lower())
```

Код файла Tasks/task_2/print_flights/__init__.py

```
from .module import print_flights


__all__ = ["print_flights"]
```

Код файла Tasks/task_2/print_flights/module.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-



def print_flights(flights):
    print("\nСписок рейсов:")


    for flight in flights:
        print(
            flight["destination"],
            flight["flight_number"],
            flight["plane_type"],
        )
```

Код файла Tasks/task_2/find_flights/__init__.py

```
from .module import find_flights
```

```
__all__ = ["find_flights"]
```

Код файла Tasks/task_2/find_flights/module.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-



def find_flights(flights, plane_type):
    found = False

    for flight in flights:
        if flight["plane_type"].lower() == plane_type.lower():
            print(
                f"Пункт назначения: {flight['destination']}, "
                f"номер рейса: {flight['flight_number']}"
            )
            found = True

    if not found:
        print("Рейсов с таким типом самолета нет."
```

Код файла Tasks/task_2/__init__.py

```
from .find_flights import find_flights
from .input_flights import input_flights
from .print_flights import print_flights
from .sort_flights import sort_flights

__all__ = [
    "find_flights",
    "input_flights",
    "print_flights",
    "sort_flights",
]
```

Код файла Tasks/task_2/main.py

```
#!/usr/bin/env python3
```

```
# -*- coding: utf-8 -*-

from .find_flights import find_flights
from .input_flights import input_flights
from .print_flights import print_flights
from .sort_flights import sort_flights

if __name__ == "__main__":
    flights = input_flights()

    sort_flights(flights)
    print_flights(flights)

    plane_type = input("\nВведите тип самолета для поиска: ")
    find_flights(flights, plane_type)
```

Приложение 3. Программный код автоматических тестов

Код файла tests/test_task_1.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from Tasks.task_1 import closure


def test_name_substitution():
    message = closure("Уважаемый %F% %N%!")
    assert message("Иванов", "Иван") == "Уважаемый Иванов Иван!"


def test_repeated_placeholders():
    message = closure("%F% / %F% / %N%")
    assert message("Петров", "Петр") == "Петров / Петров / Петр"
```

```
def test_template_without_placeholders():
    message = closure("Привет!")
    assert message("Иванов", "Иван") == "Привет!"



def test_different_names():
    message = closure("%N% %F%")
    assert message("Сидоров", "Сергей") == "Сергей Сидоров"
    assert message("Морозова", "Анна") == "Анна Морозова"
```

Код файла tests/test_task_2.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from Tasks.task_2 import (
    find_flights,
    input_flights,
    print_flights,
    sort_flights,
)



def sample_flights():
    return [
        {
            "destination": "Сызрань",
            "flight_number": "3",
            "plane_type": "ЛП-02",
        },
        {
            "destination": "Москва",
            "flight_number": "12",
            "plane_type": "ТУ-13",
        },
```

```
        {
            "destination": "Город",
            "flight_number": "1",
            "plane_type": "ТУ-1",
        },
    ]


def test_sort_flights():
    flights = sample_flights()
    sort_flights(flights)

    assert [flight["destination"] for flight in flights] == [
        "Город",
        "Москва",
        "Сызрань",
    ]


def test_sort_empty_list():
    flights = []
    sort_flights(flights)
    assert flights == []


def test_input_flights(monkeypatch):
    responses = iter(["1", "Москва", "12", "ТУ-13"])
    monkeypatch.setattr(
        "builtins.input",
        lambda _: next(responses),
    )

    assert input_flights() == [
```

```
        {
            "destination": "Москва",
            "flight_number": "12",
            "plane_type": "ТУ-13",
        }
    ]


def test_print_flights(capsys):
    print_flights(sample_flights())
    output = capsys.readouterr().out

    assert "Список рейсов:" in output
    assert "Москва 12 ТУ-13" in output


def test_find_flights(capsys):
    find_flights(sample_flights(), "лп-02")
    output = capsys.readouterr().out

    assert "Пункт назначения: Сызрань, номер рейса: 3" in output


def test_find_flights_not_found(capsys):
    find_flights(sample_flights(), "Боинг")
    output = capsys.readouterr().out

    assert "Рейсов с таким типом самолета нет." in output
```